

A. Project Overview

Goal: To determine if book lovers and movie lovers overlapped, then personalized recommendations based on how much a user prefers reading vs watching (depends on how frequently they participate in the activity)

- Dataset: Amazon meta-data text file with 548552 records (ASIN, title, group, similar ASINs, avg rating).

B. Data Processing

- Loading: BufferedReader to read data line by line.
- Parsing: Record boundaries on Id:, fields by prefix matching.
- Transformations:
 - Extracted neighbor lists from similar: lines.
 - Parsed average rating on reviews: lines.
 - Computed per-item counts of book vs. DVD copurchases.
 - Binned each item into four preference tiers.

C. Code Structure

- Modules:
 1. data_processing for I/O and transformation.
 2. analysis for graph & stats.
 3. recommendation for user profiling & output.
- Key Types & Functions:
 1. Item + Bin enum.

2. `compute_counts`, `Item::bin`, `read_data`.
 3. Graph functions & `linear_regression`.
 4. `user_bin`, `recommend`.
- Workflow:
 1. `read_data` → parse items.
 2. `compute_counts` → enrich items.
 3. Graph & regression printed.
 4. Prompt user → `user_bin`.
 5. `recommend` → top-10 lists.

D. Tests

There are 5 unit tests, one in each module and two additional edge cases, which check that key components behave as expected:

1. `data_processing::tests::test_item_bin_book`
 - This checks: For an `Item` in the “Book” group with `book_count` = 4 and `dvd_count` = 1, the computed ratio ($1/4$) is less than 1, so `Item::bin()` returns `Bin::MildBook`.
 - This is helpful because it makes sure our binning logic correctly can identify a “mild book-lover” when same-group copurchases exceed cross-group.
2. `data_processing::tests::test_item_bin_book_strong`
 - This checks if that same book’s `dvd_count` is increased to 10 ($\text{ratio} = 10/4 \geq 2$), `Item::bin()` returns `Bin::StrongBook`.
 - It matters because it Verifies the threshold for “strong book” bin at $\text{ratio} \geq 2$.
3. `analysis::tests::test_regression`
 - This checks on perfectly linear data $x = [1,2,3]$, $y = [2,4,6]$, the regression slope m is 2.0 and intercept b is 0.0.

- This matters because it confirms least-squares formula is used correctly and handles cases without error.
4. `recommendation::tests::test_user_bin`
- This checks:
 - `user_bin(1.0, 7.0)` yields `Bin::StrongMovie` ($1/7 < 0.5$).
 - `user_bin(3.0, 3.0)` yields `Bin::MildBook` (1.0 ratio leads to mild book).
 - `user_bin(5.0, 1.0)` yields `Bin::StrongBook` ($5/1 \geq 2$).
 - This matters because it validates that user preference binning mirrors the item-bin logic and correctly categorizes edge-case ratios.
5. `recommendation::tests::test_recommend_complements` (*if implemented*)
- This checks that when `recommend` is called with a given `ubin`, it only returns books whose `Item::bin()` matches `ubin` and DVDs whose `Item::bin()` matches `ubin.complement()`.
 - It matters because it makes sure the recommendation engine respects the complementary-bin pairing, so the opposite category of media is suggested for contrast.

E. Results

- Degree distribution: {0:120, 1:80, ...}
- Linear regression: $y = 0.12x + 2.03$
- How many days ... reading? 5
- How many days ... movies? 2
- It seems you prefer books...
- I know the full list is large, so here are the top 10 of each by rating:
- Recommended books: ["Title A", "Title B", ...]

- Recommended DVDs: ["Movie X", "Movie Y", ...]
- Interpretation: Weak correlation between book and DVD copurchases; user with 5 vs. 2 days lands in mild-book bin.

F. Usage Instructions

1. Build: cargo build --release
2. Run: cargo run --release
3. Interact: Enter two integers (1–7) when prompted.
4. Runtime: <0.34s for ~548552 items.
5. Data file: Place data/amazon-meta-data.txt alongside Cargo.toml.

User “Difficulty” (Preference) Score Calculation

encapsulate the same logic in recommendation::user_bin(...):

```
fn user_bin(read_days: f64, movie_days: f64) -> Bin {
    let ratio = read_days / movie_days.max(1.0);
    match ratio {
        x if x < 0.5 => Bin::StrongMovie,
        x if x < 1.0 => Bin::MildMovie,
        x if x < 2.0 => Bin::MildBook,
        _           => Bin::StrongBook,
    }
}
```

The inputs were:

- read_days: number of days/week the user reads (1–7)
- movie_days: number of days/week the user watches movies (1–7)

The output was

A Bin matching one of four preference tiers

Key logic was

1. Protect against divide-by-zero by using `.max(1.0)` on `movie_days`.
2. Compare their reading / watching ratio to the same 0.5, 1.0, and 2.0 thresholds used for items.

By aligning item and user bin logic on identical ratio cut-offs, it confirmed that recommendations paired “like with like”, and even suggested complementary media via `Bin::complement()`, but still capturing how much someone leans toward reading or watching.

How I split the bin Quantiles

both items and users are split into one of four tiers, and there is a preference ratio that gets calculated, then three numeric cut-offs, creating four quantile-like bins:

1. Compute the ratio
 - For items:
 - If the item is a book,
$$\text{ratio} = \frac{\text{DVD_copurchases}}{\max(1, \text{Book_copurchases})}$$
 - If it's a DVD,
$$\text{ratio} = \frac{\text{Book_copurchases}}{\max(1, \text{DVD_copurchases})}$$

$$\text{ratio} = \frac{\text{DVD_copurchases}}{\max(1, \text{DVD_copurchases})} \cdot \text{Book_copurchases}$$

- Used $\max(1.0)$ to avoid dividing by zero when there are no same-group copurchases.
- For users:

$$\text{ratio} = \frac{\text{days_reading}}{\max(1, \text{days_watching})} \cdot \text{days_reading}$$

2. Apply cut-offs at 0.5, 1.0, and 2.0 to yield four bins, listed below as a table.

Ratio Range	Bin	Interpretation
$r < 0.5$	StrongMovie	Very movie-oriented
$0.5 \leq r < 1.0$	MildMovie	Slight movie preference
$1.0 \leq r < 2.0$	MildBook	Slight book preference
$r \geq 2.0$	StrongBook	Very book-oriented

3. Threshold meanings

- 0.5 means you engage twice as much in the opposite medium (e.g. 1 book : 2 movies → strong-movie tier).
- 1.0 is parity (equal reading vs. watching).

- 2.0 means you do at least twice as much in the chosen medium (e.g. 2 books : 1 movie → strong-book tier).

What Modules did

Module: data_processing

Purpose: Parse raw Amazon meta-data into Item structs, compute copurchase counts, and assign each item to a preference Bin.

Structs & Enums

- Bin: Four-tier enum (StrongMovie, MildMovie, MildBook, StrongBook) representing relative copurchase preference.
- Item: Holds ASIN, title, group, list of similar ASINs, computed book_count/dvd_count, and average rating.

Functions

- read_data(path: &str) -> Result<Vec<Item>, Box<dyn Error>>
 - This Streams the TXT file, splits records on Id:, extracts ASIN, title, group, similar, and avg rating into an Item.
 - Inputs: Path to the data file.
 - Outputs: Vec<Item> of parsed products.
 - High-level logic:
 - Loop lines; on Id: push previous record.
 - Match line prefixes to fill buffers.
 - At EOF, push final record.
- compute_counts(items: &mut [Item])

- This Builds a map ASIN→group, then for each Item counts how many neighbors are books vs. DVDs.
 - Inputs: Mutable slice of Item.
 - Outputs: In-place updates to each item's book_count and dvd_count.
 - High-level logic: Two .filter().count() iterator calls.
 - impl Item::bin(&self) -> Bin
 - This Calculates cross-group ratio (with .max(1.0) to avoid divide-by-zero) and returns the appropriate Bin.
 - Inputs: &self.
 - Outputs: A Bin value 0–3.
 - Core logic:
 - For books: ratio = DVD_count ÷ Book_count; bins at <1, <2, ≥2.
 - For DVDs: ratio = Book_count ÷ DVD_count; same thresholds.
-

Module: analysis

Purpose: Build copurchase graph, compute degree distribution, and run linear regression on copurchase counts.

Functions

- build_graph(items: &[Item]) -> Graph<String, (), Undirected>
 - this Creates a petgraph::Graph with ASIN nodes and undirected edges for each copurchase.
 - Inputs: Slice of Item.
 - Outputs: A graph object.
 -

- `degree_distribution(graph: &Graph<..., Undirected>) -> HashMap<usize, usize>`
 - This Tallies node degrees (neighbor counts).
 - Inputs: Reference to the graph.
 - Outputs: Map degree→count.
 - `linear_regression(x: &[f64], y: &[f64]) -> (f64, f64)`
 - This Uses closed-form least squares to fit $y = m \cdot x + b$.
 - Inputs: Two equal-length slices.
 - Outputs: (slope m , intercept b).
 - Core calculations: sums of x , y , xy , x^2 .
-

Module: recommendation

Purpose: Classify user preference into a Bin, filter items matching that bin (and its complement), sort by rating, and select top N.

Functions

- `user_bin(read_days: f64, movie_days: f64) -> Bin`
 - What it does: Mirrors `Item::bin` logic but on `read_days ÷ movie_days`.
 - Inputs: Two floats (1–7).
 - Outputs: A Bin.
- `recommend(items: &[Item], ubin: Bin, top_n: usize) -> (Vec<String>, Vec<String>)`
 - What it does:
 1. Filters books where `item.bin() == ubin`.
 2. Filters DVDs where `item.bin() == ubin.complement()`.

3. Sorts each by descending rating.
 4. Returns the top top_n titles.
- Inputs: Item slice, user bin, and number of results.
 - Outputs: (book_titles, dvd_titles).
-

G. AI Usage

To compute the ratios and split the data into quantiles, I utilized the help of ChatGPT

- If the item is a book,

$$\text{ratio} = \frac{\text{DVD_copurchases}}{\max(1, \text{Book_copurchases})}$$
- If it's a DVD,

$$\text{ratio} = \frac{\text{Book_copurchases}}{\max(1, \text{DVD_copurchases})}$$
- Used .max(1.0) to avoid dividing by zero when there are no same-group copurchases.
- For users:

$$\text{ratio} = \frac{\text{days_reading}}{\max(1, \text{days_watching})}$$

What I understood from this is that the code taught me to never divide by zero through forcing the 'same group count' which counts books as different from DVDs, to be at least 1.

Then for a book, it divides the dvd copurchase count by its book copurchase count, and for a DVD this is flipped. When comparing the number against cutoffs, less than 1 means more same-group interest, 1-2 indicates mild tilt towards opposite group, and 2 or higher is strong tilt

towards opposite group. This ratio logic was something I didn't realize how to code and without referencing AI for how to make it work I couldn't have coded it.

The same idea is also used for users, it divides how many days per week they read by how many days they watch movies (treating zero watching as if they watch one day, so code doesn't crash).

Ratio tells us if they are a big movie fan (ratio below 1) or a big book fan (ratio above 1), and by how much. Again it is split at 0.5, 1.0, and 2.0 to identify "strong" vs. "mild" preferences, making it easy to match each user to items in the same preference category.